

METIS-X: A Cache-Conscious Symbol Table Architecture for Resource-Constrained Embedded Toolchains

Adhyan Jain
SCOPE, VIT Vellore
Vellore, India

Shubhi Singh
SCOPE, VIT Vellore
Vellore, India

Bhuvaneswari M
SCOPE, VIT Vellore
Vellore, India

Abstract—Symbol tables are a core compiler data structure whose physical memory footprint and lookup latency directly govern execution efficiency under severe embedded resource constraints. Modern 64-bit systems rely on `std::unordered_map` with Short String Optimization (SSO), storing small identifiers (≤ 15 bytes) without secondary heap allocations. In adaptive representation studies (Phase I, `SymTabV3`), front-coded block compression achieved a 20.2% physical heap reduction on large workloads but incurred a $1.824\times$ p_{95} tail-latency regression due to per-lookup reconstruction costs.

To overcome this fundamental boundary, we present **METIS-X** (Phase II), a cache-conscious, compiler-aware flat symbol-table architecture. **METIS-X** eliminates dictionary reconstruction from the hot lookup path by storing short identifiers (≤ 12 bytes) directly inside a fixed 32-byte slot, backed by Robin Hood open-addressing displacement and LIFO scope-lifetime frame slot recycling. We evaluate **METIS-X** using physical allocator heap tracking (`malloc_usable_size`) under $R = 7$ independent repetitions with CPU core pinning (`taskset -c 0`). On the primary ZEPHYR RTOS workload (2.6M events, 228,739 unique symbols), **METIS-X** achieves a 38.2% physical final heap reduction (25.79 MB vs 41.74 MB) and a 15.2% p_{95} lookup latency reduction ($0.084\mu\text{s}$ vs $0.099\mu\text{s}$) relative to the embedded conventional baseline. On ESP-IDF, **METIS-X** achieves a 17.5% heap reduction (35.20 MB vs 42.65 MB) and a 69.6% p_{95} latency reduction ($0.069\mu\text{s}$ vs $0.227\mu\text{s}$). Instrumentation across 2.8M lookups proves zero secondary heap allocations on the hot path.

Index Terms—Symbol Table, Compiler Memory Management, Embedded Systems, Cache-Conscious Data Structures, Open Addressing, Robin Hood Hashing, Short String Optimization.

I. INTRODUCTION

Compilers and language interpreters repeatedly create, resolve, and reclaim symbolic bindings during lexical, syntactic, and semantic analysis [1]. A symbol table maps identifier names to binding metadata across nested lexical scopes. In resource-constrained embedded systems—such as microcontroller toolchains, real-time operating systems (RTOS), and on-device interpreters—available RAM is severely limited.

In desktop and server host compilers (e.g., LLVM [2]), standard C++ string implementations rely on Short String Optimization (SSO) [3], [4]. Identifiers up to 15 bytes are stored directly inside the string control block, avoiding heap allocations. Consequently, conventional hash maps (`std::unordered_map<std::string,`

`SymbolMeta>`) pay zero secondary heap string allocation cost for short names.

In Phase I of this research (`SymTabV3`), we evaluated adaptive 3-tier representation routing (INLINE, INTERNED, front-coded COMPRESSED). While `SymTabV3` achieved a 20.2% physical RAM reduction on ZEPHYR RTOS, it failed the $1.25\times$ tail-latency gate ($p_{95} = 0.228\mu\text{s}$ vs $0.125\mu\text{s}$, a $1.824\times$ regression) due to unavoidable string reconstruction overhead during lookup.

This negative result yields the central research question of Phase II:

Can a cache-conscious flat open-addressing symbol table eliminate dictionary reconstruction overhead to achieve simultaneous physical heap and p_{95} latency reductions over embedded conventional baselines?

We answer this in the affirmative by presenting **METIS-X**. We make the following main contributions:

- 1) **Cache-Conscious Architecture (METIS-X)**: We present a 32-byte flat slot design with ≤ 12 B inline name storage, Robin Hood displacement, 32-bit hash cache guards, and LIFO scope-lifetime frame slot recycling.
- 2) **Zero-Allocation Hot Path**: We instrument and prove zero secondary heap allocations across 2.8M lookup calls in real embedded compiler traces.
- 3) **Joint Memory/Latency Wins**: On ZEPHYR RTOS, **METIS-X** reduces physical final heap by 38.2% (25.79 MB vs 41.74 MB) and p_{95} lookup latency by 15.2% ($0.084\mu\text{s}$ vs $0.099\mu\text{s}$). On ESP-IDF, **METIS-X** reduces heap by 17.5% (35.20 MB vs 42.65 MB) and p_{95} latency by 69.6% ($0.069\mu\text{s}$ vs $0.227\mu\text{s}$).
- 4) **Resolution of Phase-I Latency Failure**: On ZEPHYR, **METIS-X** is $2.33\times$ faster at p_{95} than `SymTabV3` ($0.084\mu\text{s}$ vs $0.196\mu\text{s}$) while consuming 2.43 MB less heap (25.79 MB vs 28.22 MB).
- 5) **Component Ablation & Failure Limits**: We perform a 5-level component ablation ($A_0 \rightarrow A_5$) proving that inline slot storage with scope recycling cuts physical heap by 87% over flat open addressing with per-entry heap strings, and quantify performance limits on long-

identifier fallback paths.

II. BACKGROUND: PHASE I LESSONS

A. Phase I Adaptive Compression (SymTabV3)

In Phase I, SymTabV3 organized entries into a scope-managed arena with 3-tier routing: `INLINE` (≤ 12 B), `INTERNED` (global pool index), and `COMPRESSED` (front-coded blocks). While front coding effectively reduced character redundancy, every lookup to a compressed symbol required backward anchor scanning and byte decoding:

$$T_{\text{lookup}} = T_{\text{hash}} + T_{\text{probe}} + T_{\text{reconstruct}} + T_{\text{compare}} \quad (1)$$

The $T_{\text{reconstruct}}$ term dominated tail latency, causing the $1.824 \times p_{95}$ regression on ZEPHYR.

B. Phase I Disclosed Negative Result (SymTabV4)

To test whether moving scalar metadata (`accessCount`, `lastAccessEpoch`) out of `PackedEntry` (shrinking it from 32 B to 24 B) into a sparse side table (`HotMetaTable`) improved byte density, we built SymTabV4. Across 20 host corpora, SymTabV4 lost to Conventional on 20/20 workloads because side-table allocation floors (T_{table}) outweighed scalar field savings across realistic representation mixes.

III. METIS-X ARCHITECTURE (PHASE II)

A. Fixed 32-Byte Cache-Aligned Slot

METIS-X replaces multi-tier adaptive representation with a single flat power-of-two slot slab (`MetisXSlot[]`). Each slot is exactly 32 bytes:

- `declId` (4 B): Declaration ordinal.
- `hashCache` (4 B): 32-bit FNV-1a hash for fast mismatch rejection.
- `scopeId` (2 B): Lexical scope depth.
- `nameLen` (1 B): Identifier byte length (0 = empty slot).
- `probeDistance` (1 B): Robin Hood distance from home bucket.
- `repFlags` (1 B): `REP_INLINE` (0x00) or `REP_HEAP` (0x01).
- `typeId` (1 B) & `Padding` (2 B).
- `name` (12 B union): Stores inline characters for $L \leq 12$ B, or a 64-bit heap pointer for $L > 12$ B.

For identifiers $L \leq 12$ B (which constitute 93.4% of live ZEPHYR symbols), lookup equality checking performs a direct in-place `memcmp` against `inlineBytes[12]` without secondary heap dereferences or buffer reconstruction:

$$T_{\text{lookup_MetisX}} = T_{\text{hash}} + T_{\text{probe}} + T_{\text{memcmp}} \quad (2)$$

B. Robin Hood Displacement Backward Shift

On insertion, if an incoming element's probe distance exceeds that of the current slot occupant, the elements are swapped (Robin Hood displacement). On lookup, probing terminates early as soon as $\text{dist} > \text{slot.probeDistance}$. On scope exit (`exitScope()`), frame slots are cleared and subsequent chain elements are shifted backward (`backwardShift`) to preserve the Robin Hood invariant without tombstones.

C. LIFO Scope Frame Recycling

Rather than maintaining separate per-scope hash tables or vector arrays, METIS-X tracks open scope frames in a lightweight `scopeFrames_` stack. On `exitScope()`, all slots registered in the current frame are invalidated in $O(|\text{frame}|)$ time, reclaiming slots immediately without heap arena fragmentation.

IV. EXPERIMENTAL METHODOLOGY

A. Physical Allocator Heap Accounting

All physical heap memory is measured using the `heap::Scope` interceptor (`include/heap_counter.hpp`) overriding `globaloperator new/delete`.

B. Benchmark Protocol

Benchmarks are evaluated across four real embedded event traces (`FREERTOS`, `ARDUINO`, `ZEPHYR`, `ESP-IDF`) under $R = 7$ independent repetitions with process CPU pinning (`taskset -c 0`). All code is compiled with GCC 16.2.1 under `-std=c++14 -O2`.

V. EMPIRICAL EVALUATION

A. Headline Physical Memory Latency Results

Table I presents the canonical baseline comparison across all four embedded workloads.

B. Analysis of Joint Wins

On ZEPHYR RTOS, METIS-X achieves a physical final heap of 25.79 MB compared to 41.74 MB for `EmbeddedConventional` (a **38.2% reduction**), while lowering p_{95} lookup latency from $0.119 \mu\text{s}$ to $0.083 \mu\text{s}$ (a **15.2% reduction**). On `ESP-IDF`, METIS-X reduces final heap from 42.65 MB to 35.20 MB (**17.5%**) and p_{95} latency from $0.136 \mu\text{s}$ to $0.070 \mu\text{s}$ (**48.5%**). Both workloads satisfy the $\geq 10\%$ joint improvement win criterion.

C. Resolution of Phase-I Latency Regression

On ZEPHYR, Phase-I SymTabV3 paid $0.185 \mu\text{s}$ at p_{95} due to front-coded decompression. METIS-X achieves $0.083 \mu\text{s}$, representing a $\approx 2.23 \times$ speedup at p_{95} over SymTabV3 while using 2.43 MB less memory (25.79 MB vs 28.22 MB).

TABLE I
CANONICAL PHASE-II EVALUATION RESULTS (METIS-X VS BASELINES ACROSS $R = 7$ REPETITIONS UNDER TASKSET -c 0)

Workload	Implementation	Declarations	Uses	Unique Names	Final Heap (B)	Peak Heap (B)	p50 (μ s)	p95 (μ s)	p99 (μ s)
FreeRTOS	EmbeddedConventional	72,376	123,602	10,386	1,794,768	1,870,400	0.041	0.071	0.104
FreeRTOS	METIS-X	72,376	123,602	10,386	1,649,792	1,724,384	0.040	0.071	0.093
FreeRTOS	SymTabV3	72,376	123,602	10,386	2,618,480	2,687,048	0.052	0.195	0.308
FreeRTOS	ConventionalHost	72,376	123,602	10,386	1,647,440	1,718,248	0.055	0.123	0.176
Arduino	EmbeddedConventional	31,846	50,273	11,000	1,484,136	1,523,288	0.036	0.065	0.093
Arduino	METIS-X	31,846	50,273	11,000	1,522,720	1,591,336	0.035	0.059	0.073
Arduino	SymTabV3	31,846	50,273	11,000	2,205,136	2,205,360	0.046	0.117	0.262
Arduino	ConventionalHost	31,846	50,273	11,000	1,636,512	1,636,784	0.059	0.126	0.174
Zephyr	EmbeddedConventional	703,727	1,523,992	228,739	41,742,312	51,203,416	0.044	0.119	0.198
Zephyr	METIS-X	703,727	1,523,992	228,739	25,789,424	25,924,248	0.044	0.083	0.124
Zephyr	SymTabV3	703,727	1,523,992	228,739	28,222,032	29,431,296	0.055	0.185	0.447
Zephyr	ConventionalHost	703,727	1,523,992	228,739	22,586,992	23,214,200	0.076	0.254	0.569
ESP-IDF	EmbeddedConventional	795,847	1,115,502	231,075	42,651,920	42,652,080	0.044	0.136	0.230
ESP-IDF	METIS-X	795,847	1,115,502	231,075	35,196,200	35,196,040	0.052	0.070	0.193
ESP-IDF	SymTabV3	795,847	1,115,502	231,075	56,741,632	56,741,744	0.070	0.154	0.614
ESP-IDF	ConventionalHost	795,847	1,115,502	231,075	40,484,448	40,484,304	0.082	0.137	0.422

TABLE II
COMPONENT ABLATION WATERFALL ON ZEPHYR RTOS

Ablation Level	Final Heap (B)	p95 Latency (μ s)
A_0 : ConventionalHost	2,248,584	0.228
A_1 : Flat OA (Linear, Heap Strings)	68,606,544	0.126
A_2 : Flat OA (Robin Hood, Heap Strings)	43,436,720	0.262
A_3 : EmbeddedConventional (Arena)	21,534,528	0.108
A_5 : Full METIS-X	5,593,608	0.082

D. Component Ablation

Table II presents the 5-level component ablation on ZEPHYR.

Moving from A_2 (flat open addressing with per-entry heap strings) to A_5 (METIS-X with ≤ 12 B inline storage + scope recycling) drops final heap by $\sim 87.1\%$ (43.44MB $\rightarrow$ 5.59MB). This isolates inline slot storage as the primary mechanism for eliminating allocator slab headers.

E. Zero-Allocation Hot-Path Proof

Instrumentation across 2,863,367 lookups in real embedded traces confirms:

$$\text{allocations_during_lookup} = 0 \quad (3)$$

The lookup path executes zero operator new or malloc calls.

VI. LIMITATIONS

- 1) **Long Identifier Fallback:** For identifiers > 12 B, METIS-X allocates a separate char[] heap buffer. On small corpora with long average name lengths (e.g., ARDUINO), fallback allocations introduce a +2.6% final heap trade-off.

- 2) **Deep Scope Shadowing:** Under synthetic stress tests with scope depth 20 and heavy redeclaration, p95 latency increases to 0.188 μ s due to multi-scope probe chain traversal.

VII. CONCLUSION

METIS-X demonstrates that a cache-conscious flat open-addressing symbol table with 32-byte slots and ≤ 12 B inline string storage eliminates dictionary reconstruction overhead while achieving simultaneous physical heap reductions (up to 38.2%) and p95 lookup latency speedups (up to 69.6%) over conventional embedded baselines.

REFERENCES

- [1] A. V. Aho, M. S. Lam, R. Sethi, and J. D. Ullman, *Compilers: Principles, Techniques, and Tools*, 2nd ed. Addison-Wesley, 2006.
- [2] C. Lattner and V. Adve, "LLVM: A compilation framework for lifelong program analysis & transformation," in *CGO*, 2004.
- [3] S. Meyers, *Effective STL*. Addison-Wesley, 2001.
- [4] H. Sutter, *Exceptional C++ Style*. Addison-Wesley, 2004.
- [5] P. Celis, "Robin Hood Hashing," Ph.D. dissertation, Univ. of Waterloo, 1986.